

# Cheat Sheet for Data Analysis #3

## Exploratory Data Analysis & Visualization

### Basic Setup and Data Loading

#### Import essential libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

*Meaning:* pandas holds the table, matplotlib draws the figure, seaborn adds ready-made statistical charts on top of matplotlib. `plt` and `sns` are the conventional short names

#### Load a CSV file into a DataFrame

```
df = pd.read_csv("Diamond.csv")
```

*Meaning:* Reads data from a CSV file and stores it in a variable called `df`; the file must sit in the same folder as the notebook

#### View basic dataset information

```
print("Dataset shape:", df.shape)
df.head()
```

*Meaning:* Displays the dimensions of the dataset and the first few rows for quick inspection

#### Write down the order of an ordered category

```
colour_order = ["D", "E", "F", "G", "H", "I"]          # best to worst
clarity_order = ["IF", "VVS1", "VVS2", "VS1", "VS2"]  # best to worst
```

*Meaning:* pandas and seaborn do not know that D is a better colour than I. Without a list, bar charts are sorted by frequency and box plots by order of appearance, so the quality scale is scrambled. Write the order once at the top of the notebook and reuse it in every chart. A category with no natural order (e.g. certification) needs no list

---

### Univariate Analysis (Single Variable)

#### Histogram for numerical variables

```
plt.figure(figsize=(8, 5))
sns.histplot(df['carat'], bins=20)
plt.title('Histogram of Carat')
plt.xlabel('Weight, carat')
plt.ylabel('Number of stones')
plt.show()
```

*Meaning:* Cuts the range of values into `bins` intervals and shows how many observations fall into each one; reveals the shape, the centre, the spread and a long tail. `kde=True` would add a smooth outline on top of the bars

#### Density plot for numerical variables

```
plt.figure(figsize=(8, 5))
sns.kdeplot(df['price'], fill=True)
plt.title('Density Plot of Price')
plt.xlabel('Price, SGD')
plt.show()
```

*Meaning:* Smoothed representation of the distribution; `fill=True` shades the area under the curve. Good for comparing distributions across groups. The smoothing can push the outline slightly past the real minimum or maximum, even below zero: this is not data

#### Box/Whisker plot for numerical variables

```
plt.figure(figsize=(8, 5))
sns.boxplot(y=df['price'])
plt.title('Box Plot of Price')
plt.ylabel('Price, SGD')
plt.show()
```

*Meaning:* The line inside the box is the median; the box runs from Q1 to Q3 and holds the middle half of the data (the IQR). The whiskers reach the most extreme observation that still lies within  $1.5 \times \text{IQR}$  of the box; anything beyond is drawn as a separate dot (an outlier by the  $1.5 \times \text{IQR}$  rule). The whiskers end at the minimum and the maximum only when there are no such dots. The rule can be changed with `whis=`, e.g. `whis=3`

### Bar chart for categorical variables, in the correct order

```
plt.figure(figsize=(10, 6))
df['colour'].value_counts().reindex(colour_order).plot(kind='bar')
plt.title('Bar Chart of Colour')
plt.xlabel('Colour (D = best, I = worst)')
plt.ylabel('Number of stones')
plt.xticks(rotation=0)
plt.show()
```

*Meaning:* Shows how many observations fall into each category. `.value_counts()` sorts the bars by frequency; `.reindex(colour_order)` puts them in the order of the list instead. For a category without a natural order, leave out `.reindex()`. `rotation=0` keeps the short labels horizontal

## Bivariate Analysis (Two Variables)

### Scatter plot (Numerical vs. Numerical)

```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='carat', y='price')
plt.title('Scatter Plot: Carat vs Price')
plt.xlabel('Weight, carat')
plt.ylabel('Price, SGD')
plt.show()
```

*Meaning:* Each dot is one observation. Reveals the direction and strength of a relationship, groups of points and unusual points. A narrow rising cloud corresponds to a strong positive correlation

### Grouped box plots (Categorical vs. Numerical)

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='colour', y='price', order=colour_order)
plt.title('Price Distribution by Colour')
plt.xlabel('Colour (D = best, I = worst)')
plt.ylabel('Price, SGD')
plt.show()
```

*Meaning:* One box per category; compares the centre and the spread of a numerical variable across groups. `order=colour_order` puts the boxes in the order of the list. Without it seaborn uses the order in which the categories first appear in the file

### Violin plots (Categorical vs. Numerical)

```
plt.figure(figsize=(10, 6))
sns.violinplot(data=df, x='clarity', y='price', order=clarity_order)
plt.title('Price Distribution by Clarity (Violin Plot)')
plt.xlabel('Clarity (IF = best, VS2 = worst)')
plt.ylabel('Price, SGD')
plt.show()
```

*Meaning:* A box plot with the density outline drawn on both sides; the widest part shows where most observations of the group lie. `order=` works exactly as in the box plot. Use `plt.xticks(rotation=45)` only if the labels are long and overlap

### Contingency table with columns in the correct order

```
cert_clarity = pd.crosstab(df['certification'], df['clarity'])[clarity_order]
```

```
print(cert_clarity)
```

**Meaning:** `pd.crosstab` counts the observations in every pair of categories and sorts the columns alphabetically. Selecting `[clarity_order]` rearranges the columns in the order of the list. The table is then used for both charts below

### Stacked bar chart (Categorical vs. Categorical)

```
cert_clarity.plot(kind='bar', stacked=True, figsize=(10, 6))
plt.title('Stacked Bar Chart: Certification vs Clarity')
plt.xlabel('Certification')
plt.ylabel('Number of stones')
plt.xticks(rotation=0)
plt.legend(title='Clarity', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

**Meaning:** One bar per row of the table, cut into pieces for its columns; reveals how the composition differs across groups. Note that the pandas `.plot()` takes `figsize=` itself, so no `plt.figure()` is needed

### Heatmap of contingency table

```
plt.figure(figsize=(8, 6))
sns.heatmap(cert_clarity, annot=True, fmt='d', cmap='Blues')
plt.title('Heatmap: Certification vs Clarity')
plt.xlabel('Clarity (IF = best, VS2 = worst)')
plt.ylabel('Certification')
plt.show()
```

**Meaning:** The same counts in a colour-coded grid: the darker the cell, the more observations. `annot=True` writes the numbers in the cells, `fmt='d'` prints them as whole numbers

---

## Multivariate Analysis (Three or More Variables)

### Create a new numerical column from existing ones

```
df['price_per_carat'] = df['price'] / df['carat']
print(df[['carat', 'price', 'price_per_carat']].describe().round(2))
```

**Meaning:** Arithmetic on two columns is applied row by row and the result is stored as a new column. Here it gives the price of one carat of each stone, a third numerical variable for the charts below. `.describe().round(2)` from Cheat Sheet #2 checks the new column at once: its minimum, median and maximum

### Correlation heatmap

```
num_df = df[['carat', 'price', 'price_per_carat']]
corr_matrix = num_df.corr()

plt.figure(figsize=(7, 5))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Matrix Heatmap')
plt.show()
```

**Meaning:** Visualizes pairwise correlations between numerical variables; red means a strong positive correlation, blue a strong negative one. `vmin=-1`, `vmax=1` fix the colour scale to the full range of a correlation, so the colours mean the same thing in every chart. The diagonal is always 1

### Pair plot (Scatterplot Matrix - SPLOM)

```
sns.pairplot(df, vars=['carat', 'price', 'price_per_carat'], hue='certification')
plt.suptitle('Pair Plot: Carat, Price and Price per Carat, by Certification', y=1.02)
plt.show()
```

**Meaning:** One scatter plot for every pair of the listed columns; the diagonal shows the distribution of each column alone. `hue=` colours the dots by a categorical variable, adding one more variable to the picture. `pairplot` draws its own figure, so it takes no `plt.figure()`; `plt.suptitle()` sets a title above all panels, and `y=1.02` lifts it slightly so it does not overlap them

## Grouped scatter plot with hue

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='carat', y='price', hue='clarity', hue_order=clarity_order)
plt.title('Carat vs Price Colored by Clarity')
plt.show()
```

*Meaning:* Adds a third variable to a scatter plot using colour; reveals how the relationship varies across categories. `hue_order=` sets the order of the categories in the legend, the same way `order=` does for the axis

---

## Several Charts in One Figure

### Subplots for multiple visualizations

```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
sns.histplot(df['carat'], bins=20, ax=axes[0, 0])
sns.kdeplot(df['carat'], fill=True, ax=axes[0, 1])
sns.histplot(df['price'], bins=20, ax=axes[1, 0])
sns.kdeplot(df['price'], fill=True, ax=axes[1, 1])
plt.tight_layout()
plt.show()
```

*Meaning:* `plt.subplots(rows, cols)` creates a grid of empty panels; `axes[row, column]` picks one of them, counting from 0. `ax=` tells a seaborn function which panel to draw on. With one row, a single index is enough: `axes[0]`, `axes[1]`

### Titles and labels of one panel

```
axes[0, 0].set_title('Histogram of Carat')
axes[0, 0].set_xlabel('Weight, carat')
axes[0, 0].set_ylabel('Number of stones')
axes[0, 0].tick_params(axis='x', rotation=0)
```

*Meaning:* Inside a grid of panels, use the methods of the panel: `.set_title()`, `.set_xlabel()`, `.set_ylabel()`, `.tick_params()`. The commands `plt.title()`, `plt.xlabel()`, `plt.xticks()` only act on the last panel drawn

### A pandas chart on a chosen panel

```
df['colour'].value_counts().reindex(colour_order).plot(kind='bar', ax=axes[0])
```

*Meaning:* The pandas `.plot()` also accepts `ax=`, so pandas and seaborn charts can share one grid

---

## Visualization Parameters Explained

### Figure size and layout

```
plt.figure(figsize=(10, 6))
plt.tight_layout()
```

*Meaning:* `figsize=(width, height)` sets the size of the figure in inches; (10, 6) gives a wide landscape chart, and larger figures read better on slides. `tight_layout()` adjusts the space between panels and around long labels so that nothing overlaps

### Title options

```
plt.title('Price Distribution by Colour', fontsize=16, fontweight='bold', color='black',
loc='center')
```

*Meaning:* `fontsize` sets the text size (larger for presentations); `fontweight` is 'normal', 'bold' or 'light'; `color` accepts a name such as 'red' or a code such as '#1f77b4'; `loc` aligns the title 'left', 'center' or 'right'

### Axis label options

```
plt.xlabel('Colour', fontsize=12, labelpad=10)
plt.ylabel('Price, SGD', fontsize=12)
```

---

*Meaning:* Axis labels are usually slightly smaller than the title. `labelpad` sets the distance between the label and the axis. Always state the unit of a numerical axis

### Tick labels

```
plt.xticks(rotation=45, fontsize=10, color='black')
plt.yticks(fontsize=10)
plt.xticks([0, 5000, 10000, 15000])
```

*Meaning:* `rotation` turns the tick labels (45 degrees prevents long labels from overlapping, 0 keeps short ones horizontal); `fontsize` and `color` change their look. A list of numbers sets the exact positions of the ticks

### Grid lines

```
plt.grid(True, linestyle='--', alpha=0.7, color='gray')
```

*Meaning:* Adds grid lines that help to read values off the chart. `linestyle` is `--` (dashed), `-` (solid), `...` (dotted) or `-.` (dash-dot); `alpha` sets the transparency from 0 (invisible) to 1 (opaque)

### Overall style

```
sns.set_style("whitegrid")
```

*Meaning:* Changes the background of all following charts. The options are `'white'`, `'dark'`, `'whitegrid'`, `'darkgrid'` and `'ticks'`; `'whitegrid'` suits data-heavy charts and slides

### Legend options

```
plt.legend(title='Clarity', loc='upper left', bbox_to_anchor=(1.05, 1), fontsize=10,
frameon=False)
```

*Meaning:* `title` names the legend; `loc` places it (`'best'`, `'upper right'`, `'lower left'` and so on); `bbox_to_anchor=(1.05, 1)` moves it outside the plot to the right; `fontsize` sets the text size; `frameon` turns the box around it on or off

### Heatmap options

```
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm', vmin=-1, vmax=1, center=0)
```

*Meaning:* `annot=True` writes the values in the cells; `fmt` formats them: `'d'` for whole numbers, `'.2f'` for two decimals, `'g'` for the general format. `cmap` chooses the colour map (`'Blues'`, `'viridis'`, `'coolwarm'`); `vmin/vmax` fix the ends of the colour scale; `center=0` puts the neutral colour of a diverging map at zero

### Colour palettes

```
sns.set_palette("Set2")
```

*Meaning:* Sets the colours of all following charts. Qualitative palettes (`'Set1'`, `'Set2'`, `'Paired'`) suit categories without order; sequential ones (`'Blues'`, `'Greens'`) suit ordered data; diverging ones (`'coolwarm'`, `'RdBu'`) suit data with a natural midpoint such as zero

### More encodings in a scatter plot

```
sns.scatterplot(data=df, x='carat', y='price', hue='clarity', size='carat',
style='certification', alpha=0.6)
sns.scatterplot(data=df, x='carat', y='price', s=50)
```

*Meaning:* `hue` encodes a variable by colour, `size` by the size of the dots, `style` by the shape of the marker. `alpha` sets the transparency of the dots (useful when they overlap); `s` sets one fixed size for all dots

---

## Practical Tips

1. **Choose the chart by the type of the variables:** one numerical - histogram, density or box plot; one categorical - bar chart; numerical vs numerical - scatter plot; categorical vs numerical - grouped box or violin plot; categorical vs categorical - stacked bar chart or heatmap
2. **Put ordered categories in order** every time: `.reindex(order)` for a pandas bar chart, `order=` for

seaborn charts, `[order]` for the columns of a crosstab

3. **Every chart needs a title and axis labels with units.** A reader should understand the chart without the code
4. **Write the interpretation under the chart:** what you see, and what it means for the question being asked
5. **A difference between groups may come from another variable.** Before drawing a conclusion, check what else distinguishes the groups from each other

Remember: a chart shows what is in the data, not why. It is the fastest way to find a question worth asking, and it never replaces checking the answer with numbers.